Theory of Algorithms – Space and Time Tradeoffs

Patrick Marais
Department of Computer Science
University of Cape Town
patrick@cs.uct.ac.za





Introduction

- Idea: Trading off space for time
  - Use more memory to reduce computation cost
- Input enhancement (preprocessing)
  - Store ancillary information about the problem instance which will make subsequently solving it faster
  - Examples: sorting by counting, Horspool and Boyer-Moore's string matching
- Prestucturing
  - An initial operation to make access to the data faster
  - Examples: hashing, indexing with B-trees (covered elsewhere in CS)
- Dynamic programming fits into this category but will be considered separately

Sorting by Counting Idea

- Sort a list whose elements fall in a restricted range of integers [L … U]
- Builds a frequency table, D, that counts the number of occurrences of each element
  - With U-L+1 entries in the table
- And a distribution table derived from frequencies to tell us where to place elements
- Yields a stable sort, sorts complex data types by key

Sorting by Counting Algorithm

- Constraints:
  - Elements in [L…U], list A with n elements to be sorted
  - Move original elements into correct positions in a new list
  - Allows additional data to be attached to each elements

```
for j ← 0 to U-L do D[j] ← 0                    // init freq
for j ← 0 to n-1 do D[A[i]-L] ← D[A[i]-L]+1     // calc freq
for j ← 0 to U-L do D[j] ← D[j-1]+D[j]              // calc distrib
for i ← n-1 downto 0 do
      j ← A[i] - L
      S[D[j]-1] ← A[i]
      D[j] ← D[j]-1
return S
```

Sorting by Counting Example

- Example: A =

13	11	12	13	12	12

Array Value	11	12	13
Frequencies	1	3	2
Distribution	1	4	6

	D[0]	D[1]	D[2]
A[5] = 12	1	4	6
A[4] = 12	1	3	6
A[3] = 13	1	2	6
A[2] = 12	1	2	5
A[1] = 11	1	1	5
A[0] = 13	0	1	5

S[0]	S[1]	S[2]	S[3]	S[4]	S[5]
			12		
		12			
					13
	12				
11					
				13	

- Time Efficiency: $\Theta(n)$ - Best so far,highly restricted inputs/application

Reminder: Brute-force String Matching

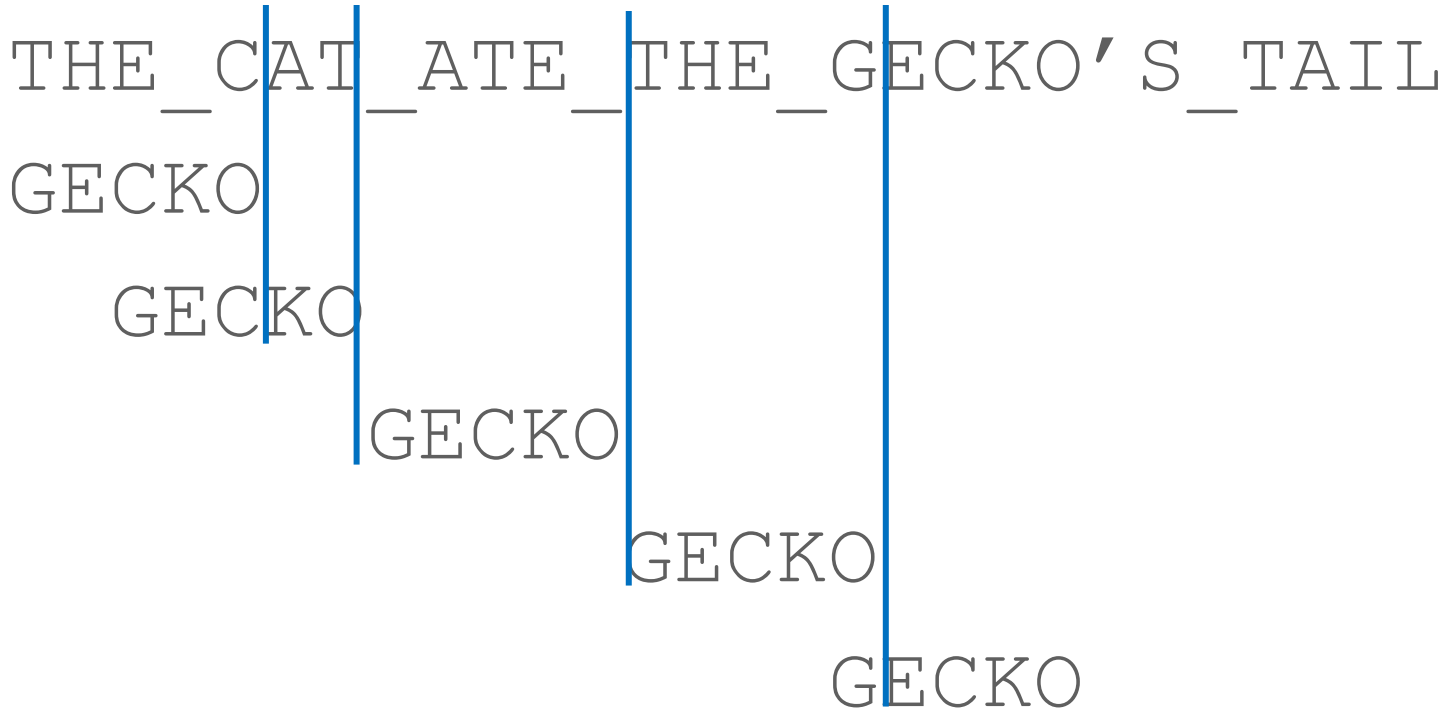
- Find the first occurrence of a pattern p in text s
  - Align pattern with start, check pattern against text until a mismatch, shift pattern one step rightward
  - If pattern matches report index
- Example:

```
THE_CAT_ATE_THE_GECKO'S_TAIL
GECKO
  GECKO
      15 shifts later
                  GECKO
```

Horspool's String Matching Idea

- **Principle**: Start matching from the end of the pattern and on a mismatch shift the pattern by more than a single space
- Use a shift table, T, to lookup shift offset on mismatch:
  - One entry per character c in alphabet
  - Gives the no. of places to shift the pattern when text character c is aligned with (opposite) the last character of the pattern
  - Maximum shift is m (the length of the pattern)

Horspool Example



- Brute Force – 17 Shifts, Horspool 5 Shifts

Horspool's Algorithm

- Align the left-most character of the pattern to the left-most character of the text (as in brute force)
- Match pattern against corresponding text from right to left (i.e. backwards through the pattern)
- If our pattern mismatches there are four cases to consider
- In all cases, compare the last letter of the pattern against the last corresponding letter in the text

Case 1

Text: `A_CAT`

Pattern: `GECKO`

`GECKO`

- If the last letter is not in the pattern: (`T` not in `GECKO`)
  Move the entire pattern past the last letter
  - i.e., shift by m

Case 2

Text: HE_FOUND

Pattern: GECKO

GECKO

- If last letters match but that letter isn't in rest of pattern Move entire pattern past the last letter
  - (exactly as in case 1)
  - i.e., shift by m

Case 3

Text: THE_C
Pattern: GECKO

GECKO

- If mismatched letter is in the pattern (C is in GECKO) but not at the end
  - Align with rightmost occurrence of that letter in the pattern
  - i.e., T(c) = m - 1- pos(c) in pattern

Case 4

Text: ENDURING

Pattern: ENGAGING

ENGAGING

- If last letters match and that letter is elsewhere in the pattern
  - Align with rightmost occurrence of that letter in pattern, ignoring the last character
  - (exactly as in case 3)

Horspool Summarised

1. Construct a shift table T
2. Align the pattern against the beginning of the text
3. Repeat until match or pattern reaches end text:
   - Starting with the last character of the pattern compare the corresponding characters in the text until either all m characters matched; or mismatch found
   - On mismatch retrieve T(c) where c is character in text aligned to last character in pattern. Shift pattern right T(c) positions.

- **Only last character in aligned text matters for shift**
- For random texts, "it can be shown" ☺ that time complexity is in Θ(n)

Example Shift Table

- Example: ENGAGING
  - m = 8
  - There are 5 unique letters

$$t(c) = \begin{cases} \text{the pattern's length } m, & \text{if } c \text{ is not among the first } m-1 \text{ characters of the pattern;} \\ \text{the distance from the rightmost } c \text{ among the first } m-1 \text{ characters} & \\ \text{of the pattern to its last character, otherwise.} & \end{cases}$$

T(E) = m – 1 – rightmost index of E = 8 – 1 – 0 = 7

T(N) = 8 – 1 – 6 = 1

T(I) = 8 – 1 – 5 = 2

T(G) = 8 – 1 – 4 = 3

T(A) = 8 – 1 – 3 = 4

T(All others) = m = 8

E	N	G	A	I
7	1	3	4	2

Example 1

```
Text:     IT_WAS_ENGAGING
Pattern: ENGAGING
```

ENGAGING

- Mismatch, last letter = E (and this is earlier in pattern)
- T(E) = 7, so shift pattern 7 places

E	N	G	A	I
7	1	3	4	2

Example 2

```
Text:     IT_WAS_VERY_ENGAGING
Pattern: ENGAGING
                   ENGAGING
```

E	N	G	A	I
7	1	3	4	2

- Mismatch, last letter = V
- T(V) = 8, (and for all character not in the pattern)

```
Text:     IT_WAS_VERY_ENGAGING
Pattern: ENGAGING
                   ENGAGING
                      ENGAGING
```

- Ater shifting by T(A) = 4

Example 3

```
Text:     IT_WAS_NOT_ENGAGING
Pattern: ENGAGING    // fail, shift T(N)=1
          ENGAGING    //fail, shift T(O) = 8
                ENGAGING //fail, shift T(I) = 2
                   ENGAGING
```

E	N	G	A	I
7	1	3	4	2

Example 4

E	N	G	A	I
7	1	3	4	2

```
Text:    IT_WAS_GOING_ENGAGINGLY
Pattern: ENGAGING //fail last in text is G, so T(G)=3
            ENGAGING
Text:    IT_WAS_GOING_ENGAGINGLY
Pattern: ENGAGING
            ENGAGING  // fail last in text is N, so T(N) = 1
             ENGAGING
Text:    IT_WAS_GOING_ENGAGINGLY
Pattern: ENGAGING
             ENGAGING // fail last in text is G, so T(G) = 3
                ENGAGING

Text:    IT_WAS_GOING_ENGAGINGLY
Pattern: ENGAGING
                   ENGAGING // fail last in text is N, so T(N) = 1
                    ENGAGING

Text:    IT_WAS_GOING_ENGAGINGLY
Pattern: ENGAGING
                    ENGAGING // fail, last in text  T(G) = 3
                       ENGAGING

Text:    IT_WAS_GOING_ENGAGINGLY
Pattern: ENGAGING
                       ENGAGING // fail, last in text  T(I) = 2
                         ENGAGING
```

Boyer-Moore Idea

- Based on same two ideas as Horspool:
  - Compare pattern characters to text from right to left
  - Given a pattern, create a shift table that determines how much to shift the pattern
- Except:
  - In addition to *bad-symbol* shift table
  - Uses a *good-suffix* shift table with same idea applied to the number of matched characters
- Efficiency:
  - Horspool's approach is simpler and at least as efficient in the average case

Bad Symbol Shift

- Rule for calculating shift is called the bad character rule
  - It needs a shift table (use Horspool as the basis)
  - Look up the shift of the rightmost mismatched char (not at the end as in Horspool)
- Example:
  - `Horspool:` `???SER`
    `BARBER`
    `BARBER`
  - `Boyer-Moore:` `???SER`
    `BARBER`
    `BARBER`
  - Since the `S` is not in `BARBER` move the pattern past the `S`

Table for BARBER

character c	A	B	C	D	E	F	. . .	R	. . .	Z	_
shift $t(c)$	4	2	6	6	1	6	6	3	6	6	6

Bad Symbol Shift

- Let m = length of string
- Let T = shift table as in Horspool
- Let k = number of matched chars from end
- Let c = mismatched char in text

  $bad_shift = max(T(c) - k , 1)$

  - Note T(c) – k could be less than 1, hence the max operation. We always can shift at least 1 position as in brute force

Difference in Shifts

```
TANSER
BARBER
    BARBER // Boyer-Moore: max(T(S)-k, 1) = max(6-2, 1) = 4
   BARBER  // Horspool: T(R)= 3
// Here Boyer-Moore beats Horspool
MY AEROPLANE
BARBER
  BARBER  // Boyer-Moore: max(T(A)-k, 1) = max(4-2, 1) = 2
   BARBER // Horspool: T(R)= 3
// Here Horspool beats Boyer-Moore
```

Preprocessing

- Before we start searching we know everything about the pattern and nothing about the text
  - The shift table is set up ONLY using information from the pattern
- Both algorithms try to extract useful information from the pattern in advance of the search, in order to maximise the size of the shift they do on each mismatch
- This is the point: The shift table is determined solely by the properties of the pattern, not the text
- It helps to understand this when we consider the next part of Boyer-Moore: Good Suffix Rule

Good Suffix Rule

- This is the second shift rule calculated by Boyer Moore
  - Uses **suff(k)** = *k* characters at end of pattern
  - Boyer Moore selects whichever shift is greater of the good suffix versus bad symbol shift rule
- Build a shift table G(k), "good suffix table", that depends on number of character matches, k
- Several Cases when building G(k):
  - Case 1 – The matching suff(k) does not occur elsewhere in the pattern
  - Case 2 – The matching suff(k) occurs earlier in the pattern
  - Case 3 – A part of the matching suff(k) occurs at the beginning of the pattern
- NOTE: we first illustrate the rationale for shift decisions, but list the actual G(k) construction algorithm after.

Case 1

- Matched suffix, suff(k), does not occur elsewhere in the pattern (k = the number of matched chars = suff(k))
- Example:
  - `Text:  . . A B A B . . .`
  - `Pattern: M A O B A B`
  - Determine G(k = 3)
  - Consider the suffix `BAB`, which is G(3); there is no earlier `BAB` in pattern
  - So here we could shift the entire pattern by m: G(3) = m = 6
- But, not always! See case 3 – if part of `BAB` occurs at start (so don't simply stop if suff(k) not seen earlier– check algorithm !)

Case 2

- The matched suffix, suff(k), occurs earlier in the pattern, with a different preceding character
- Search from right to left in pattern for another instance
- Earlier instance can overlap with the later instance
- NOTE: the character immediately left of pattern is considered too
  e.g. (x)BABOBAB where the (x) represents a 'match anything' option
  so suffix BAB actually matches the initial BAB according to Case 2 below
- Example of failure of Case 2:
  - Text: . . . . A B A B . . .
  - Pattern: O B A B O B A B
  - Shift: O B A B O B A B

  Same mismatch letter recurs in pattern so here we can shift past last match, G(k=3) = 8
- Case 2 – what Boyer-Moore actually applies:
  - Text: . . . . A B A B . . .
  - Pattern: A B A B O B A B
  - Shift: A B A B O B A B

  Different mismatch letter, so shift to align last suff(k): G(k=3) = 4

Case 3

- If we can't match entire suff(k) earlier, we don't just give up, we look for a partial match
- A part of the matching suffix occurs at the **beginning** of the pattern – find longest such match
- Must match from end of suffix
- Prefix can also overlap with the suffix
- Example:
  - `Text:  . . . . . A B A B . . .`
  - `Pattern:  A B C B A B`
  - `Shift:  A B C B A B`
  - Here the front letters (longest prefix) match part of the suffix
  - So we can only shift G(k=3) = 4

Algorithm for building G(k)

```
// Scan from right to left; pattern length = m; k = matches chars
// suff(k) is k matching characters at end of pattern

If (suff(k) is present earlier in pattern)
        if (we can find instance where character preceding it is different (or empty))
            shift to align this earlier suffix with suff(k)
            return shift

if (partial matched prefix exists)
                shift to align prefix with end of suff(k)
                return shift
shift = m
return shift
```

Boyer-Moore Algorithm

- Good-suffix shift:

  If k (> 0) symbols matched before failing, fetch d2 = *G(k)*

- Bad-symbol shift:

  If k (>= 0) symbols matched before failing, fetch

  $$d1 = \max(T(c) - k, 1)$$

  (where c is the character that didn't match)

- Algorithm:

  1. Build tables T and G
  2. When searching, use max of d1 *and* d2 (note requirement on k for use of d2)

Example 1 – Boyer Moore

Match: BAOBAB

c	A	B	C	D	. . .	O	. . .	Z	_
$t_1(c)$	1	2	6	6	6	3	6	6	6

Bad symbol table

d1 = max($T_1(c) - k, 1$)

The good-suffix table is filled as follows:

k	pattern	d_2
1	BAOBAB	2
2	BAOBAB	5
3	BAOBAB	5
4	BAOBAB	5
5	BAOBAB	5

B E S S _ K N E W _ A B O U T _ B A O B A B S

B A O B A B

$d_1 = t_1(K) - 0 = 6$

B A O B A B

$d_1 = t_1(_) - 2 = 4$

$d_2 = 5$

$d = \max\{4, 5\} = 5$

B A O B A B

$d_1 = t_1(_) - 1 = 5$

$d_2 = 2$

$d = \max\{5, 2\} = 5$

B A O B A B

Example 2 - Boyer-Moore

- For a string of **1000** 0's, S = 000….0, *how many comparisons* will Boyer-Moore matching make with the pattern P = 10000?
- Build the Tables and run the algorithm, then see what pattern emerges:

the bad-symbol table

c	0	1
$t_1(c)$	1	4

.

the good-suffix table

k	the pattern	d_2
1	10000	3
2	10000	2
3	10000	1
4	10000	5

- Now, let's run the algorithm:

```
S = 00000000.....0
    10000
```

- So, fails on last attempt = 5 comparisons
- Then, we have k = 4 matched, and d_1 = max{t_1(0)-4,1} = 1; d_2 = 5, so shift = max{1,5} = 5

```
S = 00000000.....0
    10000
        10000
```

- The *same failure pattern repeats* until we exhaust all elements in S
- We have 1000/5 = 200 possible positions (divides perfectly for |S|=1000)
- So finally, comparisons = 200*5 = 1000.

Performance – Horspool vs Boyer-Moore

- Their performance is asymptotically the same
- Need to look more closely at their operation and early opt outs to decide where they work best
- Horspool
  - Best for large alphabets with language since often a mismatch is cause by a character not in pattern, allowing maximum shift
  - E.g. English language word matching
- Boyer-Moore
  - Best for small alphabet with very long text with lots of repetition, the extra (good shift) table works to exploit repeating sub-patterns when shifting
  - E.g. DNA pattern matching (sequence of G,T, A, C)

Summary

- Preprocess all or part of the problem instance to create an acceleration data structure
- Can be very effective if the acceleration structure is used repeatedly in the subsequent algorithm
- But must consider the cost of preprocessing in the overall algorithm efficiency